DINAMICKO PLANIRANJE KONEKCIJA KOLA U RANZIRNIM STANICAMA

```
from pulp import *
solver_path = "C:\\x64_win64\\cplex.exe"
timeLimit = 300 #sekunde

#Broj dolaznih vozova
n = ["1111", "1112", "1113"]
#Broj odlaznih vozova
m =["1000", "1002", "1003"]
#vreme prispeca dolaznog voza, diskretizovano u minutama
ai = {"1111": 10, "1112": 50, "1113": 150}
#vreme pregleda dolaznog voza
ti = {"1111": 30, "1112": 30, "1113": 30}
#vreme otpreme odlaznog voza j
dj = {"1000": 100, "1002":160, "1003": 250}
#vreme potrebno za naguravanje dolaznog voza
hi = {"1111": 5, "1112": 5, "1113": 5}
#skup odsecaka koji pripadaju dolaznom vozu i
ui = {"1111": ["A", "B", "C"], "1112": ["D", "E"], "1113":
["F", "G"]}
#skup odsecaka koji mogu da budu dodeljeni odlaznom vozu
uj = {"1000": ["C", "D"], "1002": ["A", "B"], "1003":
["E", "F", "G"]}
#=============
# Presek ova dva recnika - ogranicenje 7, znaci za svaki
dolazni voz i,
# odaberi xju takvo da je u zajednicko za vozove i i j.
# Znaci za 1111 i 1000 resenje je "C"
# Za 1111 i 1002 resenje je A i B
# Za 1111 i 1003 je prazan skup
# ...za svaki i i j stampaj zajednicki odsecak ako ga ima
#za (1111, 1000) = ["C"]
#za (1111,1002) = ["A", "B"]

#skup svih odsecaka
U = ["A", "B", "C", "D", "E", "F", "G"]
#skup odlaznih vozova koji mogu da prevoze odsecak u
ju = {
    "1000": ["A","D","E","F","G"],
```

```
    "1002": ["A","B","D","E","F","G"],
    "1003": ["B","C","D","E","F","G"]
}
#osobine odsecaka
F= ["r","w","l"]
efu = {("r", "A"): 10,("w","A"): 1250, ("l","A"): 150,
       ("r", "B"): 20, ("w","B"):2350, ("l","B"):300,
       ("r", "C"):15, ("w","C"):1750, ("l","C"):225,
       ("r", "D"): 10, ("w","D"):1250,("l","D"):150,
       ("r","E"):20,("w","E"):3350, ("l","E"):300,
       ("r","F"):20, ("w","F"):2250, ("l","F"):300,
       ("r","G"):20, ("w","G"):2550, ("l","G"):300}
#r - broj kola; #w - tezina; #l - duzina;

#vrednost osobine f za odsecak u
# efu = {"A": [10,10,10], "B":[10,10,10], "C":[10,10,10],
"D":[10,10,10], "E":[10,10,10], "F":[10,10,10],
#        "G":[10,10,10]}
#minimalno vreme za odsecak da bude preusmeren na odlazni
voz nakon sto je ranziran
t = {"A": [30], "B":[30], "C":[30], "D":[30], "E":[30],
"F":[30], "G":[30]}
#vreme pregleda i formiranja odlaznog voza j
bvj = {"1000": [20, 20], "1002": [20,20], "1003": [20,20]}
#maximalne vrednosti osobine f odlaznog voza j
gjf = {("1000","r"): 2000, ("1000","w"): 2000,
("1000","l"): 2000,
       ("1002", "r"): 2000, ("1002", "w"): 2000, ("1002",
"l"): 2000,
       ("1003", "r"): 2000, ("1003", "w"):2000, ("1003",
"l"):2000}
#minimalne vrednosti osobine f odlaznog voza j
pjf = {("1000","r"): 20, ("1000","w"): 20, ("1000","l"):
20,
       ("1002", "r"): 20, ("1002", "w"): 20, ("1002",
"l"): 20,
       ("1003", "r"): 20, ("1003", "w"):20, ("1003",
"l"):20}
#trosak dodeljivanja odsecka odlaznom vozu j
```

```
#cju = {"A": [10, 10, 10], "B":[20, 20, 20], "C":[10, 15,
20], "D":[30, 30, 30], "E":[30, 30, 30], "F":[20, 20, 20],
"G":[20, 10, 20]}
cju = {("1000", "A"): 100, ("1000", "B"): 100,  ("1000",
"C"): 100, ("1000", "D"): 100, ("1000", "E"): 200,
("1000", "F"): 250, ("1000", "G"): 290,
       ("1002", "A"): 300, ("1002", "B"): 300,  ("1002",
"C"): 300, ("1002", "D"): 300, ("1002", "E"): 300,
("1002", "F"): 300, ("1002", "G"): 300,
       ("1003", "A"): 300, ("1003", "B"): 300,  ("1003",
"C"): 300, ("1003", "D"): 300, ("1003", "E"): 300,
("1003", "F"): 300, ("1003", "G"): 300}
# ,  1002: [10, 10, 10, 10, 10, 10, 10], 1003: [10, 10,
10, 10, 10, 10, 10]}
#trosak otkazivanja odlaznog voza j
oj = {"1000": 10, "1002": 20, "1003": 30}
#K - redosled ranziranja dolaznog voza
# K=[]
# for i in range(1,len(n)):
#       K.append(n.index(i))
# print(K)
#K = list(range(1, n[-1]))
#print(K)
# low=[]
# values =  demand.values()
# for val in values:
#     low.append(math.ceil(val))

K=[1,2,3] #redosled ranžiranja odgovara broju vozova
#L - redosled formiranja odlaznog voza
L= [1,2,3]#redosled formiranja odgovara broju vozova
M=1000
T=25
#==========================================
#PROMENLJIVE ODLUCIVANJA
Pik = LpVariable.dicts("Pik", [(i,k) for i in n for k in
K], 0, 1, LpBinary)
Qjl = LpVariable.dicts("Qjl", [(j,l) for j in m for l in
L], 0,1, LpBinary)
Xju = LpVariable.dicts("Xju", [(j,u) for j in m for u in
U], 0, 1, LpBinary)
```

```
Yj = LpVariable.dicts("Yj", [(j) for j in m], 0,1,
LpBinary) #1 ako nije otkazan, 0 ako je otkazan
Dlk = LpVariable.dicts("Dlk", [(l,k)  for l in L for k in
K], 0,1, LpBinary)
ROjf = LpVariable.dicts("ROjf", [(j,f) for j in m for f in
F], 0,1,LpBinary)
Rk = LpVariable.dicts("Rk", K, lowBound=0)
Sl = LpVariable.dicts("Sl", L, lowBound=0)
```

DPKK generiše sledeće osnovne odluke:

- Ranžirni niz p^i_k je 1 kada je dolazni voz **i** ranžiran kao **k-ti;**
- Niz formiranja q^l_j je 1 kada je odlazni voz j formiran kao l-ti i 0 u suprotnom;
- Formiranje odsečaka u odlazne vozove, x^u_j je 1 kada je odsečak dodeljen odlaznom vozu j, 0 u suprotnom. Važi da je:
  - $\delta_l^k=1$ ako odsečci sa dolaznog voza koji je ranžiran kao k-ti su dodeljeni odlaznom vozu koji je formiran kao l-ti, i 0 u suprotnom.
  - $\rho_j^f=0$ ako minimalni limit osobine f za odlazni voz j nije ispunjen, 1 u suprotnom.
- Formiranje odlaznih vozova, y_j, **je 1 kada je odlazni voz j formiran, 0 ako je otkazan.**
- Vremena početka k-tog ranžiranja i l-tog formiranja označenih sa r_k **i** s_l**.**

```
#=========================================
#FUNKCIJA CILJA (1)
model = LpProblem("Railcar_yards", LpMinimize)

model += (
      lpSum(cju[(j, u)]*Xju[(j, u)] for j in m for u in U)
- lpSum(oj[(j)]*Yj[(j)] for j in m)
)
```

$$\min z = \sum_{u \in U} \sum_{j \in J^u} c_j^u x_j^u - \sum_{j=1,\dots,m} o_j y_j$$

```
#https://www.guru99.com/python-list-index.html
#my_list = ['Guru', 'Siya', 'Tiya', 'Guru', 'Daksh',
'Riya', 'Guru']
# K = []
# for i in range(0, len(n)) :
#     #if my_list[i] == 'Guru' :
#         K.append(i)
# #print("Originallist ", my_list)
# #povecanje svih elemenata liste za 1:
# K=[k+1 for k in K]
# print(K)

# for k in  range(len(n)):
#       print(n[k], n[k+1]) #=> [1111,1112,1113]
```

```
# for i in range(len(voce_povrce)):
#     print("Na indeksu:", i, "nalazi se", voce_povrce[i])
# K2 = range(len(n))
# print(K2)
#OGRANICENJA
#Ogranicenje 2: hump engine can only start to push an
inbound train over the hump after
#the engine finishes the previous train
for k in K[1:]:
    model += (
        Rk[k] >= Rk[k-1] +
        lpSum(hi[i] * Pik[(i,k)] for i in n)
    )
```

$$r_{k+1} - r_k - \sum_{i=1,\dots,m} h_i p_i^k \geq 0,\ k = 1,2,\dots,n\text{-}1$$

```
#Ogranicenje 3: Dolazni vozovi ne mogu biti ranzirani pre
nego je odradjen tehnicki pregled
for k in  K:
      model += (
            Rk[k] - lpSum((ai[(i)]+ti[(i)])*Pik[(i,k)] for
i in n) >= 0
            #mora k+1 jer za indeks 0 nema vrednosti za
Pik
      )
```

$$r_k - \sum_{i=1,\dots,n} (a_i + t_i) p_i^k \geq 0,\ k = 1,\dots,n$$

```
# # #Ogranicenje 4 i 5: Dodeljuju svakom dolaznom vozu
tacno jednu poziciju u ranzirnom nizu
for k in K:
      model += (
            lpSum(Pik[(i,k)] for i in n) == 1
            #mora k+1 jer za indeks 0 nema vrednosti za
Pik
      )
for i in n:
      model += (
            lpSum(Pik[(i,k)] for k in K) == 1
      )

# #Ogranicenje 6: ranzirna lokomotiva formira bilo koji
odlazni voz samo kada svi odsecci dodeljeni
odlaznom vozu su ranzirani sa operativnom vrem rezervom
koja se zasniva na delta(lk)...
```

$$s_1 - r_k - \sum_{i=1,\dots,n} h_i p_i^k - T \geq M(\delta_1^k - 1)$$
$$k = 1,\dots,n;\ l = 1,\dots,m$$

```
for k in  K:
       for l in L:
            model += (
                   Sl[l] - Rk[k] -
lpSum(hi[(i)]*Pik[(i,k)] for i in n) - T >= M*(Dlk[(l,k)]-
1)
                  #T je skalar, nije definisano posebno za
svaki odsecak
     )
# #Ogranicenje 7: if any cuts from the k-th classified
inbound train are assigned to the l-th assembled outbound
train
for k in  K:
      for i in n:
            for j in  m:
                  for l in L:
                        model += (
                         #ovde odradi presek da nije
prazan skup izmedju dolaznih i odlaznih vozova
                              M*(Dlk[(l,k)]+2 -
Pik[(i,k)] - Qjl[(j,l)]) >= lpSum(Xju[(j,u)] for u in U if
u in ui[i] and u in uj[j])
                        )
#Ogranicenje 8:
for l in L[:-1]: #uzimam sve elemente osim poslednjeg zbog
l+1
      model+=(
            Sl[l+1]- Sl[l] - lpSum(vj[j]*Qjl[(j,l)] for j
in m)>=  0
      )
#Ogranicenje 9: svaki formirani odlazni voz zadovoljava
planirano vreme otpreme
for l in L[:-1]:
      model += (
           Sl[l] <= lpSum((dj[(j)] - vj[j] -
bj[j])*Qjl[(j,l)] for j in m) + M*(1-lpSum(Qjl[(j,l)] for
j in m))
      )
```

$$M(\delta_l^k + 2 - p_i^k - q_j^l) \geq \sum_{u \in U_i \cap U^j} x_j^u$$
$$k, i = 1,\dots,n$$
$$l, j = 1,\dots,m;\ \mathrm{U_i} \cap U^j \neq \varnothing;$$

```
# Ogranicenje 10 i 11: Svaki odlazni voz je dodeljen samo
jednoj poziciji u nizu za formiranje, pod uslovom da nije
otkazan
for j in m:
      model += (
            lpSum(Qjl[(j,l)] for l in L) == Yj[j]
      )

for l in L:
      model += (
            lpSum(Qjl[(j,l)] for j in m) == 1
      )

#Ogranicenje 12 svaki odsecak dodeljen najvise jednom
odlaznom vozu
#Mora strožije ograničenje - svaki odsečak na jedan
odlazni voz
for u in U:
      model += (
             lpSum(Xju[(j,u)] for j in ju) == 1
      )

#ZADATAK: Zašto ograničenje 13 čini da problem postane
nedopustiv?
# #Ogranicenje 13 Maksimalne i minimalne gran vrednosti
svakog odlaznog voza
# for j in m:
#       for f in F:
#             model += (
#                    lpSum(efu[(f,u)]*Xju[(j,u)] for u in
U) <= gjf[(j,f)]*Yj[(j)]
#       )

#Ogranicenje 14: Minimalne gran vrednosti...
for j in m:
      for f in F:
             model += (
                    lpSum(efu[(f,u)]*Xju[(j,u)] for u in U)
>= pjf[(j,f)]*ROjf[(j,f)]
      )
```

```
#Ogranicenje 15:
for j in m:
      model += (
            lpSum(ROjf[(j,f)] for f in F) >= Yj[(j)]
      )

# #problem solve
#model.solve()
if os.path.exists(solver_path):
  if "cplex" in solver_path:
    model.solve(apis.CPLEX_CMD(path=solver_path,
timeLimit=timeLimit))
    print("Solving with: cplex")
  elif "scip" in solver_path:
    model.solve(apis.SCIP_CMD(path=solver_path,
timeLimit=timeLimit))
    print("Solving with: scip")
else:
  model.solve(pulp.PULP_CBC_CMD(timeLimit=timeLimit))
  print("Solving with: cbc")

#Print the results
print("Status: ", LpStatus[model.status])

# for v in model.variables():
#     print(v.name, "=", v.varValue)
# print("The optimal value of the objective function is
=", value(model.objective))

# # #print(allyards[0:5])

model.writeLP('RailCarYards.txt')

for v in model.variables():
    if v.varValue is not None and abs(v.varValue) > 1e-6:
        print(v.name, "=", v.varValue)

print("=== MODEL INFO ===")
print("Broj promenljivih:", len(model.variables()))
print("Broj ograničenja:", len(model.constraints))
```